

STM32 GUI Framework

API 参考手册

目标平台：STM32F103xB + SSD1306 128x64 OLED

底层驱动：u8g2 & STM32CubeMX HAL

版本 1.0 — 2026-05

模块总览：

- menu — 多级菜单系统，支持页面切换动画
- popup — 弹窗对话框（数值调节 / 布尔开关 / Toast 通知）
- ux_move — 基于坐标插值的动画引擎
- Key — 四按键输入，软件去抖动

1. 项目概览

本框架为 STM32 平台上的单色 OLED 显示器提供完整的 GUI 工具集，构建于 u8g2 图形库 与 STM32CubeMX HAL 之上。包含多级菜单、弹窗对话框、平滑动画以及整洁的开发者 API。

1.1 硬件需求

- MCU: STM32F103xB (Cortex-M3, 20 KB RAM, 64 KB Flash)
- 显示屏: SSD1306 128x64 OLED, I2C 接口, 地址 0x78
- 按键: 4 个, 上拉输入 (PC13 / PA0 / PA1 / PA2)

1.2 文件结构

```
Core/Inc/menu.h          ██████████
Core/Src/menu.c           ██████████
Core/Inc/popup.h         ████████████████ / ███ / Toast██
Core/Src/popup.c          ██████████
Core/Inc/ux_move.h        ██████████
Core/Src/ux_move.c         ██████████
Core/Inc/Key.h            ██████████
Core/Src/Key.c             ██████████
Core/Inc/stm32_u8g2.h     u8g2 ████████
Core/Src/stm32_u8g2.c     I2C ██████████
```

1.3 构建命令

开发者编写的 .c 文件请添加到根目录 CMakeLists.txt （非 cmake/stm32cubemx/CMakeLists.txt, 后者由 CubeMX 自动生成）：

```
target_sources(${CMAKE_PROJECT_NAME} PRIVATE
    Core/Src/stm32_u8g2.c
    Core/Src/ux_move.c
    Core/Src/Key.c
    Core/Src/menu.c
    Core/Src/popup.c
)
```

配置与构建：

```
cmake --preset Debug
cmake --build build/Debug
```

2. 菜单系统 (menu.h)

2.1 数据结构

menu_item_t (菜单项)

表示菜单中的一行。

```
typedef struct menu_item {
    const char      *name;          // 名称
    void            (*action)(void); // 回调函数
    const struct menu_page *submenu; // 子菜单
} menu_item_t;
```

若 action != NULL，用户按下确认键(Key 3)时触发回调。若 submenu != NULL，按确认键时跳转至该子页面。action 优先级高于 submenu。

menu_page_t (菜单页)

表示菜单层级中的一页。

```
typedef struct menu_page {
    const char      *title; // 标题
    const menu_item_t *items; // 菜单项
    uint8_t         count; // 项数
    const struct menu_page *parent; // 父菜单 (NULL)
} menu_page_t;
```

menu_state_t (菜单运行时状态)

菜单系统的全局状态，在 main.c 中分配一个实例即可。

```
typedef struct {
    const menu_page_t *current;
    uint8_t            selected;
    anim_ctrl_t        scroll_anim; // 滚动动画
    anim_ctrl_t        bar_anim;   // 进度条动画
    int16_t            bar_target_y, bar_target_w;
    menu_trans_e       trans;      // 切换动画
    anim_ctrl_t        trans_anim; // 切换动画 0->100
} menu_state_t;
```

2.2 API 参考

函数名	描述
void menu_init(menu_state_t *s, const menu_page_t *root)	以给定的根页面初始化菜单状态。启动时调用一次。
void menu_update(menu_state_t *s)	每帧在主循环中调用，紧随 anim_manager_update() 之后。处理页面切换的收尾工作。
void menu_render(u8g2_t *u8g2, menu_state_t *s)	将当前菜单页（或切换动画）渲染至 u8g2 帧缓冲区。需要包裹在 u8g2_FirstPage / NextPage 之间。
bool menu_key_up(menu_state_t *s)	向上移动选择。有动作时返回 true。
bool menu_key_down(menu_state_t *s)	向下移动选择。有动作时返回 true。
bool menu_key_enter(menu_state_t *s)	执行 action 回调或进入子菜单。
bool menu_key_back(menu_state_t *s)	返回父页面。在根页面时无操作。

2.3 页面切换动画

用户进入子菜单或按返回键时，旧页面分裂离开（标题栏向上飞出，文字向下沉出），新页面从反方向汇聚进入。时长由 TRANS_MS 定义（默认 800 ms）。切换过程中选择条跟随新页面首行文字运动。
切换期间菜单按键被屏蔽（menu_key_* 返回 false）。切换动画由 menu_update(.) 驱动，每帧必须调用。

2.4 集成示例

```
// 1. ■■■■■■■■■■
static const menu_item_t settings_items[] = {
    {"Brightness", bright_cb, NULL},
    {"Power Save", power_cb, NULL},
};
static menu_page_t settings_page = {
    .title="Settings", .items=settings_items,
    .count=2, .parent=NULL // ■■■■■
};
static menu_item_t root_items[] = {
    {"Settings", NULL, &settings_page},
};
static menu_page_t root_page = {
    .title="Main Menu", .items=root_items,
    .count=1, .parent=NULL
};

// 2. ■■■■■■■■■■
settings_page.parent = &root_page;

// 3. ■■■■
static menu_state_t menu;
menu_init(&menu;, &root_page);

// 4. ■■■■
while (1) {
    anim_manager_update();
    menu_update(&menu;);
    int8_t k = Key();
    if (k==1) menu_key_up(&menu;);
    else if (k==2) menu_key_down(&menu;);
    else if (k==3) menu_key_enter(&menu;);
    else if (k==4) menu_key_back(&menu;);
    u8g2_FirstPage(&u8g2;); do { menu_render(&u8g2;, &menu;);
    } while (u8g2_NextPage(&u8g2;));
}
```

3. 弹窗系统 (popup.h)

三种弹窗类型共享同一个状态机 (IDLE -> OPENING -> ACTIVE -> CLOSING -> IDLE)，以及统一的滑入/滑出动画 (由 ux_move 引擎驱动)。

3.1 数值调节弹窗 (popup_num*)

允许用户通过上下键调整整数数值。显示标题、分隔线以及居中大号数字。确认键(Key 3)或返回键(Key 4)关闭弹窗。

popup_num_cfg_t

```
typedef struct {
    const char *title;    // ■■■■
    int16_t *value;       // ■■■■■■■■■■
    int16_t min;          // ■■■■■■
    int16_t max;          // ■■■■■■
    int16_t step;         // ■■■■■■■■
} popup_num_cfg_t;
```

■ ■	■ ■
void popup_num_init(popup_num_t *p)	初始化数值弹窗实例。
void popup_num_open(popup_num_t *p, const char *title, int16_t *value, int16_t min, int16_t max, int16_t step)	配置并打开弹窗，伴随滑入动画。
bool popup_num_active(const popup_num_t *p)	非 IDLE 状态时返回 true。
void popup_num_update(popup_num_t *p, int8_t key)	处理按键输入。空闲时为空操作。
void popup_num_render(const popup_num_t *p, u8g2_t *u8g2)	在菜单上方渲染。

3.2 布尔开关弹窗 (popup_bool*)

一个开关切换器。上下键均翻转布尔值。可自定义 ON/OFF 标签。

popup_bool_cfg_t

```
typedef struct {
    const char *title;
    bool *value;
    const char *text_on;    // true ■■■■■■
    const char *text_off;   // false ■■■■■■
} popup_bool_cfg_t;
```

■ ■	■ ■
void popup_bool_init(popup_bool_t *p)	初始化布尔弹窗实例。
void popup_bool_open(popup_bool_t *p, const char *title, bool *value, const char *on, const char *off)	配置并打开。
bool popup_bool_active(const popup_bool_t *p)	非 IDLE 状态时返回 true。
void popup_bool_update(popup_bool_t *p, int8_t key)	Key 1/2 = 翻转；Key 3/4 = 关闭。
void popup_bool_render(const popup_bool_t *p, u8g2_t *u8g2)	在菜单上方渲染。

3.3 Toast 通知弹窗 (popup_toast*)

紧凑型自动消失通知。无需用户交互——滑入后停留 1 秒，自动滑出。使用较小字号 (6x10) 及紧凑内边距。区别于其他弹窗：popup_toast_update(.) 没有 key 参数——关闭计时由 HAL_GetTick() 内部驱动。

4. 动画引擎 (ux_move.h)

轻量级、纯整数运算的动画引擎，用于坐标插值。最多支持 MAX_ANIM_NUM（默认 10）个并发动画，提供缓动函数、步骤序列、暂停/恢复以及反向播放功能。

4.1 anim_ctrl_t（动画控制块）

```
typedef struct {
    anim_state_t state;          // IDLE | PLAYING | PAUSED | FINISHED | BACKING
    uint32_t start_time;        // ■■ HAL_GetTick()
    uint32_t duration;          // ■■■ (ms)
    int16_t start_x, start_y, end_x, end_y;
    int16_t cur_x, cur_y;       // ■■■■■■
    void (*easing)(...);        // ■■■■■■
    void (*on_finish)(void *);  // ■■■■
    // ■■■■■■
    const anim_step_t *steps;
    uint8_t step_count, current_step;
    bool loop;
} anim_ctrl_t;
```

4.2 缓动函数

所有缓动函数共享同一个签名：

```
void easing(int32_t t,          // ■■■■■ (ms)
            int32_t b,          // ■■■■
            int32_t c,          // ■■■■ (end - start)
            int32_t d,          // ■■■■ (ms)
            int32_t *out);      // ■■■■■■
```

quad_ease_out — 二次缓出（减速），默认缓动函数。
linear_ease — 匀速线性插值。

4.3 API 参考

■■	■■
void anim_init(anim_ctrl_t *a)	重置所有字段；默认缓动为 quad_ease_out。
void anim_set_position(anim_ctrl_t *a, int16_t x, int16_t y)	将当前 / 起始 / 结束均设为 (x,y)，状态置为 IDLE。
void anim_start(anim_ctrl_t *a, int16_t sx, int16_t sy, int16_t ex, int16_t ey, uint32_t ms, void *easing)	从 (sx,sy) 动画至 (ex,ey)，时长 ms 毫秒。自动注册到全局管理器。
void anim_start_step(anim_ctrl_t *a)	从第 0 步开始播放步骤序列。
void anim_pause(anim_ctrl_t *a)	暂停；保存已流逝时间供恢复使用。
void anim_resume(anim_ctrl_t *a)	从暂停位置恢复。
void anim_stop(anim_ctrl_t *a)	强制进入 IDLE，从管理器注销。
void anim_back(anim_ctrl_t *a)	反向朝起始位置播放。
void anim_manager_update(void)	必须在主循环中每帧调用。
bool anim_manager_is_idle(void)	无已注册动画时返回 true。
void anim_get_position(anim_ctrl_t *a, int16_t *x, int16_t *y)	读取 cur_x / cur_y（内联函数）。
bool anim_is_playing(anim_ctrl_t *a)	state == ANIM_PLAYING 时返回 true（内联函数）。

4.4 常见用法

仅对 Y 轴做动画（菜单滚动常用）：

```
anim_start(&a;, 0, a.cur_y, 0, target_y, 350, quad_ease_out);
```

X 作为宽度、Y 作为位置（选择条）：

```
anim_start(&bar;, old_w, old_y, new_w, new_y, 200, quad_ease_out);  
int16_t w = bar.cur_x;    // ■■■■  
int16_t y = bar.cur_y;    // ■■ Y ■■
```


5. 按键输入 (Key.h)

四个低电平有效的按钮，内置上拉。通过“松手瞬间检测”实现软件去抖动，无需定时器滤波。

5.1 引脚映射

Key 1	PA0	(key1)	■■■■
Key 2	PA1	(key2)	■■■■
Key 3	PA2	(key3)	■■ / ■■
Key 4	PC13	(user_key)	■■

5.2 API

```
unsigned char Key(void);    // ■■■■■■■■■■■■ (1..4, 0=■■■■)
void Key_Tick(void);       // ■■ 20 ms ■■■■ (■ SysTick ISR ■)
```

5.3 ISR 接线 (stm32f1xx_it.c)

```
void SysTick_Handler(void) {
    static uint8_t counter = 0;
    if (++counter == 20) { Key_Tick(); counter = 0; }
    HAL_IncTick();
}
```

SysTick 以 1 kHz 频率触发；计数器将其分频至 50 Hz (20 ms) 供按键扫描。HAL_IncTick() 必须在 Key_Tick() 之后调用。

6. 显示集成 (stm32_u8g2.h)

u8g2 库配置为 SSD1306 128x64 通过硬件 I2C (地址 0x78) 驱动。使用全帧缓冲模式 (_f 变体), 占用 1 KB RAM。

6.1 初始化

```
// main.c ■■
u8g2_t u8g2;
HAL_TIM_Base_Start(&htim1); // I2C ■■■■■
u8g2Init(&u8g2); // ■■■■ u8g2_Setup_ssd1306..._f

// ■■■■■■■■
u8g2_FirstPage(&u8g2);
do {
    // ■■■■■■■■
} while (u8g2_NextPage(&u8g2));
```

6.2 SetFontMode

在使用 XOR 模式 (颜色 2) 之前, 务必调用 u8g2_SetFontMode(u8g2, 1)。透明模式 (1) 确保字形的背景像素不会被绘制。

6.3 本框架使用的字体

- 菜单文字: u8g2_font_helvB10_tr (Helvetica Bold 10 px, 比例宽, 透明底色)
- 标题栏: u8g2_font_6x10_tr (6x10 像素, 等宽)
- Toast: u8g2_font_6x10_tr

6.4 颜色模式

```
SetDrawColor(u8g2, 0)   ■■ (■■■■)
SetDrawColor(u8g2, 1)   ■■ (■■■■)
SetDrawColor(u8g2, 2)   XOR (■■■■■■■■)
```

选择条使用 XOR 模式: 圆角矩形的选择框以颜色 2 绘制在已画好的菜单文字之上。框与白色文字重叠处, XOR 产生黑色 (反色); 与空白处重叠, XOR 产生白色 (框体本身)。

7. 主循环完整模板

```
int main(void) {
    HAL_Init();
    SystemClock_Config();
    HAL_SYSTICK_Config(HAL_RCC_GetHCLKFreq() / 1000);
    MX_GPIO_Init(); MX_I2C1_Init(); MX_TIM1_Init();
    HAL_TIM_Base_Start(&htim1);

    u8g2_t u8g2; u8g2Init(&u8g2);

    // ■■■■■■■■
    settings_page.parent = &root_page;
    // ...

    // ■■■■■■■■
    static menu_state_t menu;
    static popup_num_t pop_num;
    static popup_bool_t pop_bool;
    static popup_toast_t toast;
    menu_init(&menu, &root_page);
    popup_num_init(&pop_num);
    popup_bool_init(&pop_bool);
    popup_toast_init(&toast);

    while (1) {
        anim_manager_update();
        menu_update(&menu);
        int8_t k = Key();

        popup_num_update(&pop_num, k);
        popup_bool_update(&pop_bool, k);
        popup_toast_update(&toast);

        if (!popup_num_active(&pop_num)
            && !popup_bool_active(&pop_bool)
            && !popup_toast_active(&toast)) {
            if (k==1) menu_key_up(&menu);
            else if (k==2) menu_key_down(&menu);
            else if (k==3) menu_key_enter(&menu);
            else if (k==4) menu_key_back(&menu);
        }

        u8g2_FirstPage(&u8g2); do {
            menu_render(&u8g2, &menu);
            popup_num_render(&pop_num, &u8g2);
            popup_bool_render(&pop_bool, &u8g2);
            popup_toast_render(&toast, &u8g2);
        } while (u8g2_NextPage(&u8g2));
    }
}
```

8. 扩展指南

新增菜单项：定义 `menu_item_t[]` 和 `menu_page_t`，在运行时连接 `parent` 指针，在父页面的 `items` 数组中新增一项并将 `submenu` 指向 `&new_page`。

新增 Action 回调：编写 `void fn(void)` 函数，设置为某个 `menu_item_t` 的 `action`。在回调内使用 `popup_num_open` / `popup_bool_open` / `popup_toast_show` 弹出对话框。

新增弹窗类型：参照 `popup.h` / `popup.c` 中的模式。定义 `init` / `open` / `active` / `update` / `render` 函数。在 `main.c` 中添加实例，并在主循环中调用其 `_update` 和 `_render`。